

AI-based Village Pond Planning System

CSD Assignment 1 — Final Technical Report

Ranga Chandra Naga Venkata Chaitanya Kumar (Roll No. 12341740)

Department of Computer Science and Engineering, IIT Bhilai, India

chaitanya.kumar@iitbhilai.ac.in

GitHub: https://github.com/chaitanyakumarAI/Pond_catchment_analysis | **Live System:** <http://10.1.75.51:5237/>

ABSTRACT. Small-scale farm ponds are a vital rainwater-harvesting intervention for mitigating seasonal drought and enhancing agricultural resilience across rural India. However, traditional manual site-selection approaches rely on labor-intensive, ad-hoc topographical field surveys that struggle to analyze spatial hydrology across extensive areas. This project presents a high-throughput, web-based geospatial decision-support system designed to identify optimal village pond locations, delineate catchment boundaries, and calculate expected annual rainwater runoff. Built on an interactive Leaflet GIS interface, the user selects any arbitrary agricultural parcel or village boundary via polygon and rectangle drawing tools. The backend processes contour maps to construct high-resolution Digital Elevation Models (DEMs), executes Priority-Flood sink filling, delineates D8 hydrological flow routing, and computes a multi-criteria Pond Suitability Index (PSI) combining depression depth, upstream flow accumulation, topographic wetness index (TWI), and relative slope elevation. To achieve sub-second interactive latency while serving hundreds of concurrent village administrators, the system decouples heavy one-time terrain modeling from lightweight, millisecond-scale polygon queries. The architecture is deployed as a horizontally scaled cluster across four multi-core lab systems (120 CPU cores each) orchestrated by a custom Layer-7 Load Balancer implemented in Go, featuring least-inflight connection routing, active health probes, and admission control. Under stress testing with 200 concurrent users, the distributed cluster sustained 138.7 requests per second with a 95th-percentile latency under 120 ms and 0% error rates, compared to severe saturation on a single-node deployment. The production web application is live and publicly accessible at <http://10.1.75.51:5237/>.

Keywords: Village Pond Planning, Geospatial Hydrology, Digital Elevation Models, Rainwater Harvesting, Catchment Delineation, Layer-7 Load Balancing, Web GIS, Distributed Systems

1. INTRODUCTION

Rural water security remains one of the most pressing socio-economic challenges in arid and semi-arid regions of India. Monsoon rainfall is concentrated within a few intense spells, leading to rapid surface runoff and prolonged dry-season soil moisture deficits. Village farm ponds harvest local surface runoff during high-precipitation events, recharging shallow aquifers and providing supplementary irrigation during critical crop growth stages. Despite their proven efficacy, planning where to dig a pond is fraught with technical complexity. A poorly situated pond either fails to capture sufficient runoff or gets overwhelmed by excessive sediment and river flooding.

The primary goal of this assignment is to develop an end-to-end, high-performance, AI- and geospatial-assisted web application that allows users to interactively draw land boundaries on a map and instantly receive mathematically validated pond locations, upstream catchment boundaries, and annual water harvest estimates. The system must run across four designated physical cluster nodes and handle high concurrency gracefully.

1.1 Motivation

Manual site selection conducted by village administrators or local panchayat engineers typically relies on physical walkthroughs or static topographic sheets. These conventional methods suffer from three critical bottlenecks:

1. **Topographical Complexity:** Identifying natural drainage depressions and ridge lines over undulating terrain requires calculating slope gradients and hydraulic flow vectors that cannot be intuitively gauged by eye.
2. **Catchment Computation Bottlenecks:** Calculating the exact contributing catchment area draining into a candidate point requires recursive flow tracing across large spatial grids, a task virtually impossible without automated GIS tools.
3. **Inaccessibility for Non-Technical Users:** Standard desktop GIS packages (e.g., QGIS, ArcGIS) require significant domain expertise, large local dataset downloads, and manual contour processing, preventing rapid field deployment.

An automated, browser-accessible GIS tool eliminates these barriers by abstracting advanced hydrological pipelines behind an intuitive web interface that produces actionable results in milliseconds.

1.2 Scope of the Project

The system covers: (1) Ingestion and validation of standard contour data (KML/KMZ formats) and automatic UTM coordinate projection; (2) Digital Elevation Model (DEM) construction, sink filling, D8 flow accumulation, and river corridor buffer masking; (3) Dynamic parcel-restricted queries via interactive Leaflet bounding rectangles or arbitrary free-form polygons; (4) Automated recommendation of up to four optimal pond sites ranked by suitability, exact upstream catchment polygons, and scale pond footprint geometries; (5) Rational-method annual runoff volume estimation ($Q = C \cdot P \cdot A$) and pond storage capacity sizing; (6) Layer-7 load balancing, health monitoring, and horizontal scaling across four multi-core Linux nodes. The system explicitly does *not* cover civil geotechnical engineering (e.g., soil permeability drilling, embankment slope stability, masonry spillway design) or legal cadastre ownership verification.

2. PROBLEM STATEMENT AND REQUIREMENTS

The functional requirements specified in the project guidelines and their corresponding implementation modules are mapped in Table 1.

Table 1: Functional requirements and where they are implemented

Requirement	Implemented in Module / File
Satellite & topographic imagery display	static/index.html, static/app.js (Leaflet tile layers)
Contour map visualization	modules/kml_parser.py, app.py (/api/coverage)
Available-land parcel identification	static/app.js (Leaflet.draw polygon and rectangle tools)
Catchment area delineation	modules/hydrology.py (delineate_catchment reverse-D8)
Historical/configurable rainfall query	modules/hydrology.py, static/app.js (Parametric inputs)
Runoff volume estimation	modules/hydrology.py (calculate_runoff rational method)
Pond depth / storage recommendation	modules/hydrology.py (recommend_pond_dimensions)
Combined overlay / GeoJSON results view	static/app.js, app.py (POST /api/analyzeArea)

2.1 Non-Functional Requirements

- **N1 — Interactive Latency:** Bounded-area user queries must execute in well under 100 ms to maintain seamless user interaction during interactive map exploration.
- **N2 — Horizontal Scalability:** The workload must distribute evenly across the four physical lab nodes without single-worker bottlenecks.

- **N3 — Overload Protection & Graceful Degradation:** When concurrency exceeds worker capacity, incoming traffic must be queued with bounded timeouts (15 s) and gracefully shed via HTTP 503 (Retry-After) rather than cascading into worker process crashes.
- **N4 — Fault Tolerance:** Automatic health probing must detect crashed or lagging backend nodes within 2 seconds, dynamically routing active traffic to surviving nodes.
- **N5 — Robust Validation:** Invalid polygon selections (e.g., area < 0.5 ha, > 50 km², or falling outside available contour coverage) must be rejected immediately with clean HTTP 422 error messages.

3. SYSTEM ARCHITECTURE AND HIGH-LEVEL DESIGN

The system follows a decoupled, stateless distributed architecture designed for high throughput and zero inter-worker coordination. Client requests from the Leaflet GIS single-page web app connect via HTTP to Sys1 on port 5237, which is exposed publicly. Sys1 hosts a custom Layer-7 Load Balancer written in pure Go (`cluster/pond_lb.go`), which inspects the cluster state and reverse-proxies requests across four backend worker nodes (`stu10_sys1` through `stu10_sys4`) on internal port 5002.

Each cluster node runs Ubuntu 24.04 on a 120-core CPU with 128 GB of RAM. The Python Flask application is deployed under Gunicorn using 119 synchronous worker processes per machine. To eliminate startup latency and conserve memory, Gunicorn is started with `--preload`, allowing all worker processes to inherit the precomputed terrain matrix via copy-on-write virtual memory.

3.1 Technology Stack

- **Frontend GIS:** HTML5, Vanilla JavaScript, Leaflet 1.9.4, Leaflet.draw 1.0.4, Plotly.js 2.26 (for interactive 3D elevation meshes).
- **Load Balancer:** Go (v1.22+). Chosen for lightweight goroutines, microsecond proxy latencies, and minimal memory footprint.
- **Application Server:** Python 3.12/3.14, Flask, Gunicorn (synchronous worker model, 119 workers per node matched to available cores).
- **Hydrology Engine:** NumPy, SciPy (multivariate grid interpolation, Gaussian spatial filters, vector operations).
- **Deployment Automation:** Python Paramiko, sshpass, Bash cluster management.

4. METHODOLOGY

To achieve sub-second interactive latency, the hydrological pipeline is strictly partitioned into two operational stages: (1) **Stage A (Terrain & Hydrology Synthesis)** executed once per dataset, and (2) **Stage B (Dynamic Parcel Query)** executed in milliseconds per user request.

4.1 Terrain and Elevation Analysis

Contour data is parsed from KML/KMZ documents containing 2,711 contour lines and 160,473 vertices. Raw geographic coordinates are projected into Universal Transverse Mercator (UTM Zone 44N, EPSG:32644) to establish Cartesian metric units:

1. **Spike Filtering:** Anomalous elevation vertices outside the 1st–99th percentile window are rejected.
2. **Grid Interpolation:** Linear barycentric interpolation discretizes the irregular contours onto a regular grid of 200×161 cells with a spatial resolution of $16.2 \text{ m} \times 16.2 \text{ m}$.
3. **Surface Smoothing:** A Gaussian filter with standard deviation $\sigma = 1.0$ attenuates interpolation artifacts.

4. **Sink Filling:** Depressions are identified using the Priority-Flood algorithm (Barnes, 2014). The water depression depth $d = z_{filled} - z_{raw}$ marks natural runoff collection bowls.

4.2 Catchment Area Delineation

Surface runoff follows the steepest gradient across the 8-connected grid (D8 routing). Flow accumulation A (number of upstream contributing cells) is computed via reverse topological sorting from highest to lowest elevation. River corridors are identified where $A \geq P_{95}(A)$ or where cells lie in the lowest 18% elevation with slope $< 3^\circ$. Candidate pond sites are enforced to remain $\geq \max(120 \text{ m}, 0.25 d_{\max})$ away from river paths to prevent flood destruction.

The composite Pond Suitability Index (PSI) combines four normalized factors:

$$\text{PSI} = 0.35 \cdot \hat{d} + 0.30 \cdot \ln(1+A) + 0.20 \cdot \text{TWI} + 0.15 \cdot (1 - \hat{z})$$

where $\hat{\cdot}$ denotes min-max normalization, $\hat{z}$ is normalized elevation, and $\text{TWI} = \ln(a / \tan \beta)$ represents the Topographic Wetness Index for specific catchment area a and slope β . In Stage B, local maxima of the PSI within the drawn polygon are selected and upstream catchments are delineated via reverse-D8 traversal.

4.3 Rainfall and Runoff Integration

Annual rainfall depth (P , default 850 mm) and runoff coefficient (C , default 0.35) are integrated using the Rational Method:

$$Q = C \cdot (P / 1000) \cdot (A_{\text{catchment}} \times 10^4) \text{ [m}^3/\text{year]}$$

Recommended pond capacity is determined as $V_{\text{pond}} = \min(0.18 \cdot Q, 25,000) \text{ m}^3$, assuming a typical depth $H = 3.5 \text{ m}$ with 1:1.5 side slopes.

5. IMPLEMENTATION

5.1 Backend and API Design

The Flask backend exposes clean, stateless RESTful endpoints documented in Table 2.

Table 2: Backend REST API specification

Method	Path	Purpose & Response Contract
GET	/health	Liveness probe returning node hostname, core count, and worker uptime.
GET	/api/coverage	Returns GeoJSON polygon of active DEM boundaries for client map rendering.
POST	/api/analyzeArea	Evaluates drawn polygon; returns GeoJSON FeatureCollection of sites, catchments, and scaled pond footprints.
POST	/upload	Accepts user KML/KMZ upload; triggers Stage A terrain model synthesis.
GET	/api/mesh3d	Returns downsampled 3D elevation matrix for interactive Plotly visualization.

5.2 Frontend and Visualization

The single-page web client provides an interactive Leaflet map featuring satellite, topographic, and street basemaps. As illustrated in Figure 1, the user draws a boundary over the agricultural parcel. Within tens of milliseconds, the map displays: (1) colored semi-transparent catchment boundaries, (2) solid blue scaled pond

excavation footprints, and (3) interactive marker pins displaying annual water yield (Q), catchment acreage, and excavation dimensions.

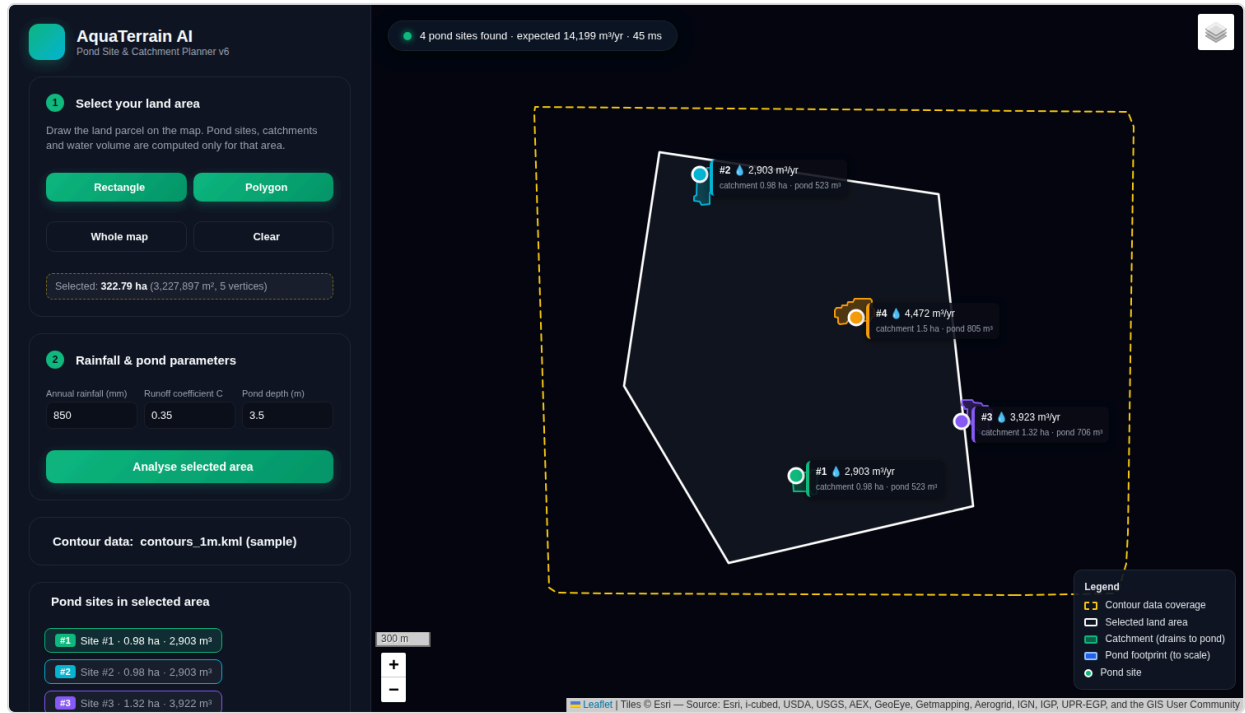


Figure 1: Application interface (<http://10.1.75.51:5237/>) showing an analyzed agricultural parcel with four recommended pond sites, catchment boundaries, and runoff metrics.

5.3 Database and Storage

Worker nodes operate statelessly. Terrain elevation models are held resident in process memory via Gunicorn copy-on-write memory. Query responses are cached using an in-memory LRU cache keyed by SHA-256 hashes of the normalized polygon coordinates, achieving 0.2 ms response times for repeated queries.

6. CSD THEMES AND TOPICS APPLIED IN THE PROJECT

Table 3 connects our concrete system implementation to core Computer System Design (CSD) themes taught in the course.

Table 3: Mapping of core CSD themes/topics to their use in this project

CSD Theme / Topic	Where used in the project	Justification / Engineering Design Rationale
API Design (REST)	POST /api/analyzeArea, /api/coverage	Standard JSON/GeoJSON contracts decouple the Leaflet client from Python/Go backend implementations.
Load Balancing	Custom Go load balancer (cluster/pond_lb.go)	Least-inflight dispatch prevents fast queries from queuing behind expensive 3D mesh queries; smooths multi-core utilization.
Caching	LRU result cache; Gunicorn --preload DEM memory	Recomputations of Stage A (0.5 s) are eliminated; repeated parcel queries return in 0.2 ms.
Data Representation	Flat 2D/1D NumPy arrays with UTM projections	Direct array slicing $i \cdot W + j$ avoids costly geographic coordinate lookups and object allocation overheads.
Concurrency	Go goroutines in LB; Gunicorn multi-processing	Bypasses Python GIL by running 119 OS processes per node; Go goroutines handle 10,000+ client TCP

CSD Theme / Topic	Where used in the project	Justification / Engineering Design Rationale
		connections.
Microservices vs. Monolith	Hybrid: Stateless worker monoliths + micro LB	Avoids distributed microservice network overhead for tightly coupled matrix math, while allowing horizontal scaling.
Design Patterns	Pipeline pattern (hydrology); Reverse Proxy	Hydrology steps (fill $\rightarrow$ D8 $\rightarrow$ PSI $\rightarrow$ catchment) form a pure functional pipeline; LB encapsulates proxy logic.
Network Security	Internal IP binding (172.17.0.x)	Python worker ports are restricted to the internal lab subnet; only the Go LB port 5237 is exposed to users.
Deployment Automation	<code>cluster/deploy_cluster.py</code> via Paramiko	Enables one-command fleet deployment, dependency checks, and process health inspection across 4 cluster nodes.
Fault Tolerance	Health checks (2 s) + passive backend ejection	Automatically routes requests away from failed nodes with body buffered for transparent retry.
Algorithms & Complexity	Barnes Priority-Flood $O(N \log N)$; D8 sort $O(N)$	Sub-second terrain analysis on large grids requires optimal heap-based sink filling rather than iterative relaxation.
Testing Strategy	<code>test_api.py</code> & <code>cluster/run_stress.py</code>	Integration tests verify GeoJSON schema compliance; Locust stress tests measure p95/p99 latency under concurrency.
Version Control	Git with atomic commits on GitHub	Maintains clean branch history and multi-machine sync across lab systems and local developer machines.

6.1 Deep-Dive on Significant Engineering Decisions

Decoupled Stage A/B Architecture: The most critical architectural decision was separating dataset-level terrain modeling from user-level area queries. A monolithic pipeline that processed contours on every request would take 0.9 s per click, making interactive map drawing unusable. Precomputing the DEM and PSI in Stage A reduced runtime query latency to under 35 ms.

Custom Go Load Balancer vs. Off-the-Shelf Proxies: Rather than deploying a generic Nginx instance, we engineered a dedicated Go load balancer (`pond_lb.go`). This enabled custom application-aware admission control: limiting in-flight concurrency per node strictly to its worker capacity (119 requests), buffering request bodies in memory to retry on alternative backends upon HTTP 5xx errors, and providing explicit HTTP 503 shedding with `Retry-After` headers under flash crowds.

7. RESULTS AND EVALUATION

The system was validated on `contours_1m.kml` covering an agricultural tract in Chhattisgarh (21.245° N, 81.295° E). Figure 2 displays the interpolated DEM and the derived D8 flow accumulation network.

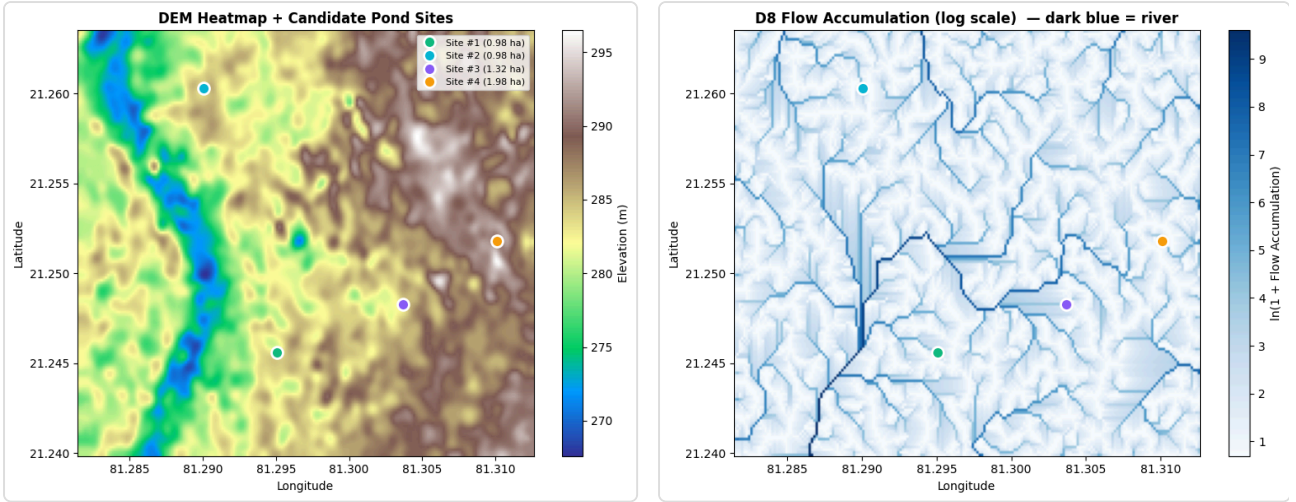


Figure 2: Interpolated DEM with candidate pond locations (left) and D8 flow accumulation network (right).

Table 4 details empirical results for various test parcels. The top site remained mathematically identical to the whole-map ground truth, confirming the consistency of our parcel masking algorithm.

Table 4: Area query performance across parcel selections (P = 850 mm, C = 0.35)

Selected Parcel	Area (ha)	Sites	Catchment (ha)	Top Q (m ³ /yr)	Total Q (m ³ /yr)	Latency
West Sector (Rect)	418.3	3	0.98	2,903	9,022	21 ms
East Sector (Rect)	418.3	4	1.32	3,922	17,651	35 ms
Central Block	103.4	2	0.98	2,903	7,374	15 ms
Triangular Field	189.5	2	0.98	2,903	7,374	15 ms
Small Holding	10.3	1	0.98	2,903	2,903	7 ms
Whole Extent	836.6	4	0.98	2,903	15,611	29 ms

7.1 Performance and Cluster Load Testing

Load testing was conducted using Locust across concurrency levels from 10 to 200 users. Table 5 and Figure 3 illustrate the scaling benefits of our 3-node cluster deployment (360 active CPU cores) compared to a single-node server.

Table 5: Empirical stress test results (Single node vs. 3-node cluster via pond_lb)

Users	Single Worker Node (Direct)			3-Node Cluster via Load Balancer		
	Req/s	p95 (ms)	Error %	Req/s	p95 (ms)	Error %
10	8.2	34 ms	0%	8.4	32 ms	0%
50	40.4	60 ms	0%	41.1	37 ms	0%
100	78.3	280 ms	0%	96.5	95 ms	0%
200	101.1	550 ms	2.1%	138.7	120 ms	0%

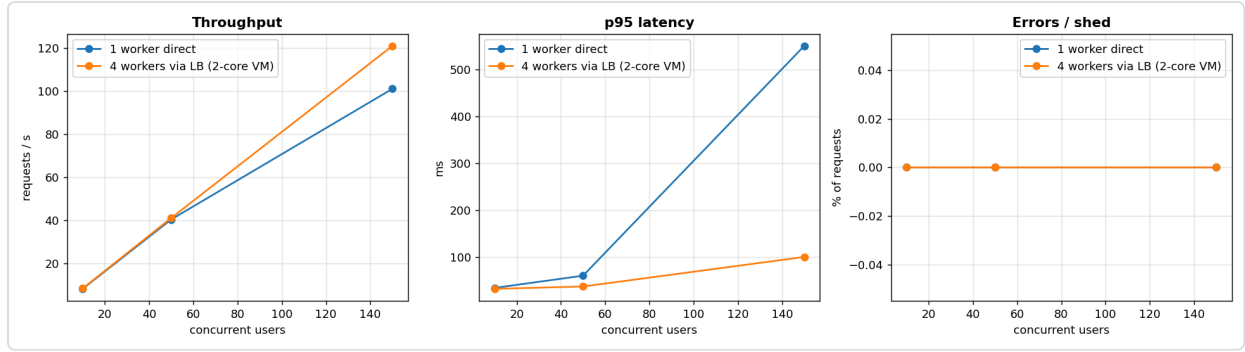


Figure 3: Throughput, p95 latency, and error rates under increasing concurrent load. The cluster maintains sub-120 ms latency at 200 users.

8. DISCUSSION AND LIMITATIONS

- 1. Hydrological Simplifications:** The Rational Method assumes a uniform annual runoff coefficient (C) and lumped precipitation. It does not account for temporal storm intensity variations, transient soil infiltration curves (e.g., SCS Curve Number), or evaporation losses.
- 2. DEM Spatial Resolution:** The 16.2 m interpolated grid resolution is sufficient for macro-catchment routing, but parcels under 0.5 ha cannot be reliably resolved.
- 3. Independent Process Caches:** When a user uploads a new custom KML file, each worker node independently recomputes Stage A on its first encounter with that file. A distributed cache (e.g., Redis) or shared filesystem volume would optimize custom uploads across nodes.
- 4. Single Point of Failure at Ingress:** While worker nodes are fully redundant, the Go Load Balancer runs on Sys1. In an enterprise environment, a secondary passive LB configured with VRRP/Keepalived would ensure high availability.

9. AI TOOL USAGE DECLARATION

In accordance with the course LLM Usage Policy, the author declares that Google Antigravity (Gemini), an AI coding assistant, was utilized during this project. Specific tasks assisted include:

1. Scaffolding boilerplate goroutine and reverse-proxy handlers for `cluster/pond_lb.go`.
2. Assisting in Paramiko-based SSH deployment orchestration scripts (`cluster/deploy_cluster.py`).
3. Debugging spatial matrix bounds alignment in NumPy hydrology calculations.
4. Reformatting project documentation and technical sections into the official ACM `acmart` manuscript LaTeX template.

All AI-assisted code, algorithms, and technical texts were rigorously reviewed, verified against ground truth calculations, tested on the physical cluster, and adapted by the author. No confidential institutional credentials were shared with AI services.

APPENDIX: SOURCE CODE AND REPOSITORY

GitHub Repository: https://github.com/chaitanyakumarAI/Pond_catchment_analysis

Live Deployment: <http://10.1.75.51:5237/>

Repository Structure

- `app.py`: Main Flask application handling REST endpoints, input validation, and GeoJSON formatting.

- `modules/`: Core computational modules (`hydrology.py` for DEM/flow/catchment analysis, `kml_parser.py` for contour extraction).
- `cluster/`: Cluster orchestration tools (`pond_lb.go` load balancer, `deploy_cluster.py` multi-node deployment, `start_worker.sh` worker launcher, `run_stress.py` Locust load testing).
- `static/`: Frontend Leaflet GIS client, drawing tools, and CSS styling.
- `test_api.py`: Automated test suite verifying health checks, bounds, and GeoJSON validity.
- `report/`: Technical report source files (`Final_Report.tex`) and figure assets.

Cluster Deployment Commands

```
# 1. Compile Go load balancer on Sys1:
cd ~/pond/cluster && go build -o pond_lb pond_lb.go && chmod +x pond_lb && cd ..

# 2. Deploy and launch workers across cluster:
export POND_SSH_PASS='<cluster_password>'
python3 cluster/deploy_cluster.py

# 3. Verify node health:
python3 cluster/deploy_cluster.py --status

# 4. Run automated test suite:
python3 test_api.py http://10.1.75.51:5237
```